SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA0704

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. *(BL4,BL5)*

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based

on overall link quality rather than just the shortest distance.Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

RUBRICS FOR CALCULATION

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

ANSWERS

1. Network Throughput and Packet Loss

The question requires throughput, packet-loss calculation, validation, classification, and a complete performance summary.

Pseudocode:

```
ALGORITHM NetworkPerformanceAnalysis

BEGIN

   // Input values
   totalDataReceived ← 8000          // bits
   time ← 4                          // seconds
   packetsSent ← 200
   packetsReceived ← 180

   // Validation
   IF time <= 0 THEN
      DISPLAY "Error: Time must be greater than zero."
      STOP
   END IF

   IF packetsSent <= 0 THEN
      DISPLAY "Error: Total packets sent must be greater than zero."
      STOP
   END IF

   IF packetsReceived < 0 OR packetsReceived > packetsSent THEN
      DISPLAY "Error: Invalid packet values."
      STOP
   END IF

   // Calculate throughput
   throughput ← totalDataReceived / time

   // Calculate packet loss
   packetsLost ← packetsSent - packetsReceived
   packetLossPercentage ← (packetsLost / packetsSent) × 100

   // Display calculated results
   DISPLAY "----- NETWORK PERFORMANCE -----"
   DISPLAY "Total Data Received: ", totalDataReceived, " bits"
   DISPLAY "Time: ", time, " seconds"
   DISPLAY "Packets Sent: ", packetsSent
   DISPLAY "Packets Received: ", packetsReceived
   DISPLAY "Packets Lost: ", packetsLost
   DISPLAY "Throughput: ", throughput, " bps"
   DISPLAY "Packet Loss: ", packetLossPercentage, "%"

   // Network classification
   IF throughput > 1500 AND packetLossPercentage < 10 THEN
      classification ← "Efficient"
   ELSE
      classification ← "Inefficient"
   END IF

   DISPLAY "Network Classification: ", classification

   // Complete summary
   DISPLAY "----- PERFORMANCE SUMMARY -----"

   IF classification = "Efficient" THEN
      DISPLAY "The network provides good throughput"
```

```
    DISPLAY "and maintains packet loss below the required limit."
  ELSE
    DISPLAY "The network does not satisfy the required"
    DISPLAY "throughput and packet-loss conditions."
  END IF

END
```

Expected Result:

Throughput:
8000 / 4 = 2000 bps

Packets Lost:
200 - 180 = 20

Packet Loss:
(20 / 200) × 100 = 10%

Therefore:

- Throughput = **2000 bps**
- Packet loss = **10%**
- Required throughput: **> 1500 bps**
- Required packet loss: **< 10%**
- **Classification = Inefficient**

Important: Even though throughput is good, the packet loss is exactly **10%**, while the question requires it to be **less than 10%**. Therefore, the correct classification is **Inefficient**.

2. Bandwidth Utilization Monitoring

The question specifically expects periodic monitoring, storing bandwidth values in an array/list, looping through all links, detecting utilization above 80%, and recommending a backup link.

Pseudocode:

```
ALGORITHM BandwidthMonitoring

BEGIN

  // Store network link names
  links ← ["Link1", "Link2", "Link3", "Link4", "Link5"]

  // Array to store bandwidth utilization
  utilization ← [0, 0, 0, 0, 0]

  // Backup link information
  backupLink ← "Backup_Link"

  DISPLAY "Starting Network Bandwidth Monitoring..."

  // Continuous monitoring
  WHILE TRUE DO

    DISPLAY "----- NEW MONITORING CYCLE -----"

    // Collect utilization of every link
    FOR i ← 0 TO LENGTH(links) - 1 DO

      utilization[i] ← GET_BANDWIDTH_UTILIZATION(links[i])

      DISPLAY links[i], " Utilization: ",
          utilization[i], "%"

      // Check for overloaded link
```

```
        IF utilization[i] > 80 THEN

            DISPLAY "WARNING: ", links[i],
                " is overloaded."

            DISPLAY "Recommendation: Switch traffic to ",
                backupLink

            SWITCH_TRAFFIC(links[i], backupLink)

            DISPLAY "Traffic redirected successfully."

        ELSE

            DISPLAY links[i],
                " is operating within acceptable limits."

        END IF

    END FOR

    DISPLAY "Monitoring cycle completed."

    // Wait before next monitoring cycle
    WAIT 10 seconds

  END WHILE

END
```

How it supports efficient bandwidth management:

This algorithm continuously monitors every network link and stores its utilization values in an array. If a link exceeds **80% utilization**, it is considered overloaded and traffic is redirected to a backup link. This prevents excessive congestion and helps maintain better network performance. Periodic monitoring also allows the administrator to respond to changing traffic conditions instead of relying on fixed bandwidth decisions.

Why this is Level 4:

This answer includes:

- **Input:** network links and utilization
- **Output:** utilization status, overload warning, backup recommendation
- **Loop:** continuously monitors all links
- **Condition:** checks utilization > 80%
- **Array:** stores utilization values
- **Action:** switches traffic to backup link
- **Periodic execution:** uses a monitoring interval
- **Complete handling:** both normal and overloaded links are handled

3. Best Communication Path Based on Link Quality

This is the most important question because it requires **arrays/lists, periodic monitoring, a link-quality score, path selection, overload detection, failure detection, alternate-path selection, and administrator alerts**.

Link Quality Formula:

The question does **not prescribe an exact formula** for the link-quality score. Therefore, to make the algorithm complete, we can define a normalized scoring approach where:

- Higher bandwidth → better
- Lower delay → better
- Lower utilization → better

One suitable formula is:
Link Quality Score =
0.5 × Normalized Bandwidth
+ 0.3 × Inverse Normalized Delay
+ 0.2 × Inverse Normalized Utilization
This gives the greatest importance to bandwidth while still considering delay and congestion.

Pseudocode:

```
ALGORITHM DynamicBestPathSelection

BEGIN

    // Network paths between headquarters and branches
    paths ← ["Path1", "Path2", "Path3", "Path4", "Path5", "Path6"]

    // Arrays for link metrics
    bandwidth[6]
    delay[6]
    utilization[6]
    status[6]
    qualityScore[6]

    monitoringInterval ← 10 seconds

    DISPLAY "Starting Dynamic Network Path Monitoring..."

    // Continuous monitoring
    WHILE TRUE DO

        DISPLAY "================================="
        DISPLAY "NEW NETWORK MONITORING CYCLE"
        DISPLAY "================================="

        // Step 1: Collect current network metrics
        FOR i ← 0 TO LENGTH(paths) - 1 DO

            bandwidth[i] ← GET_BANDWIDTH(paths[i])
            delay[i] ← GET_DELAY(paths[i])
            utilization[i] ← GET_UTILIZATION(paths[i])
            status[i] ← GET_LINK_STATUS(paths[i])

        END FOR

        // Step 2: Calculate link quality score
        FOR i ← 0 TO LENGTH(paths) - 1 DO

            IF status[i] = "FAILED" THEN

                qualityScore[i] ← 0

            ELSE

                normalizedBandwidth ←
                    bandwidth[i] / MAX(bandwidth)

                inverseDelay ←
                    MIN(delay) / delay[i]

                inverseUtilization ←
                    (100 - utilization[i]) / 100

                qualityScore[i] ←
                    (0.5 × normalizedBandwidth) +
                    (0.3 × inverseDelay) +
                    (0.2 × inverseUtilization)
```

```
    END IF

END FOR

// Step 3: Detect overloaded or failed links
FOR i ← 0 TO LENGTH(paths) - 1 DO

    IF status[i] = "FAILED" THEN

        DISPLAY "ALERT: ", paths[i], " has FAILED."

        SEND_ALERT_TO_ADMIN(
            paths[i], "Link Failure"
        )

    ELSE IF utilization[i] > 80 THEN

        DISPLAY "ALERT: ", paths[i],
                " is overloaded."

        SEND_ALERT_TO_ADMIN(
            paths[i], "Link Overload"
        )

    END IF

END FOR

// Step 4: Select path with highest valid quality score
bestPath ← NONE
highestScore ← -1

FOR i ← 0 TO LENGTH(paths) - 1 DO

    IF status[i] ≠ "FAILED" AND
       utilization[i] <= 80 THEN

        IF qualityScore[i] > highestScore THEN

            highestScore ← qualityScore[i]
            bestPath ← paths[i]

        END IF

    END IF

END FOR

// Step 5: Handle situation where no normal path is available
IF bestPath = NONE THEN

    DISPLAY "WARNING: No normal path available."

    // Search for the best remaining alternate path
    FOR i ← 0 TO LENGTH(paths) - 1 DO

        IF status[i] ≠ "FAILED" THEN

            IF qualityScore[i] > highestScore THEN

                highestScore ← qualityScore[i]
                bestPath ← paths[i]

            END IF

        END IF
```

```
        END FOR

        IF bestPath ≠ NONE THEN

            DISPLAY "Alternate path selected: ",
                  bestPath

            SEND_ALERT_TO_ADMIN(
               bestPath,
               "Alternate path activated"
            )

        ELSE

            DISPLAY "CRITICAL: All network paths failed."

            SEND_ALERT_TO_ADMIN(
               "NETWORK",
               "All communication paths unavailable"
            )

        END IF

    ELSE

        DISPLAY "Best Communication Path: ", bestPath
        DISPLAY "Best Path Quality Score: ", highestScore

    END IF

    // Step 6: Display complete path information
    DISPLAY "----- PATH PERFORMANCE SUMMARY -----"

    FOR i ← 0 TO LENGTH(paths) - 1 DO

        DISPLAY paths[i]
        DISPLAY "Bandwidth: ", bandwidth[i]
        DISPLAY "Delay: ", delay[i]
        DISPLAY "Utilization: ", utilization[i], "%"
        DISPLAY "Status: ", status[i]
        DISPLAY "Quality Score: ", qualityScore[i]

    END FOR

    // Step 7: Wait for the next monitoring cycle
    WAIT monitoringInterval

  END WHILE

END
```

Explanation:

This algorithm continuously collects **bandwidth, delay, utilization, and link status** for every network path and stores them in arrays. A quality score is then calculated so that a path with **higher bandwidth, lower delay, and lower utilization** receives a better score.

The algorithm selects the path with the **highest valid quality score**, rather than simply selecting the shortest path. If the selected path becomes overloaded above **80% utilization** or fails, the algorithm searches for another available path and generates an administrator alert.

This improves:

1. Network Reliability
Failed or overloaded paths are detected automatically, allowing traffic to use another available path.

2. Routing Efficiency
The best path is selected using actual network conditions instead of only physical distance.

3. Fault Tolerance
If the preferred path fails, an alternate path is selected automatically.

4. Adaptability
Because bandwidth, delay, and utilization are collected repeatedly, the selected path can change according to changing network conditions.

5. Congestion Prevention
Avoiding links above 80% utilization reduces the possibility of severe network congestion.